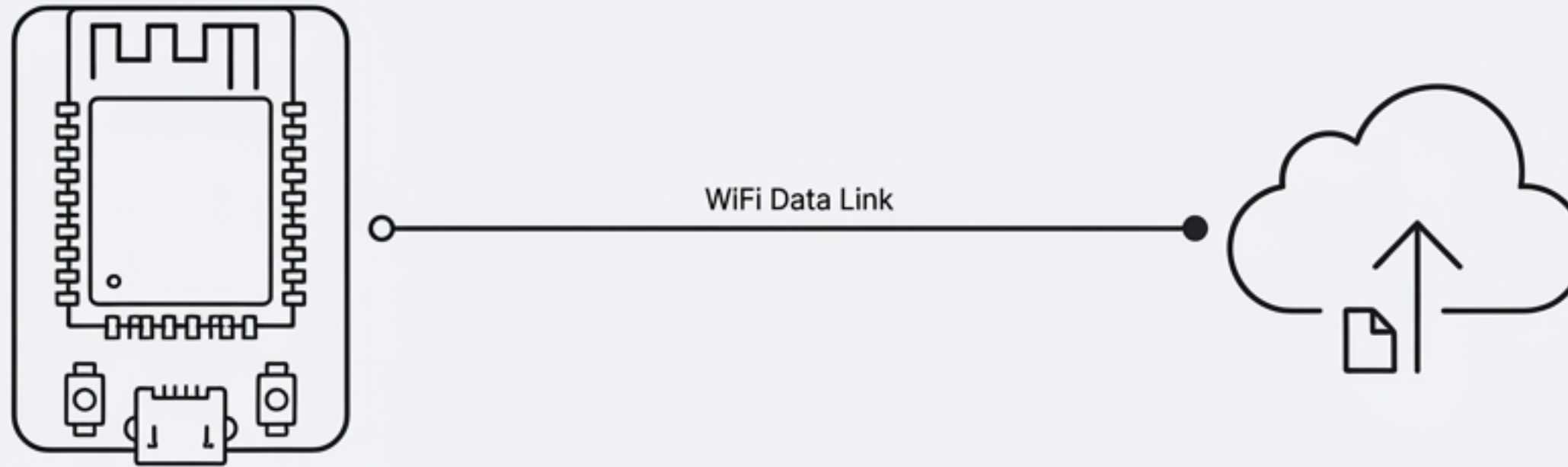


ESP32 Communication Protocols

Mastering HTTP GET and HTTP POST Requests



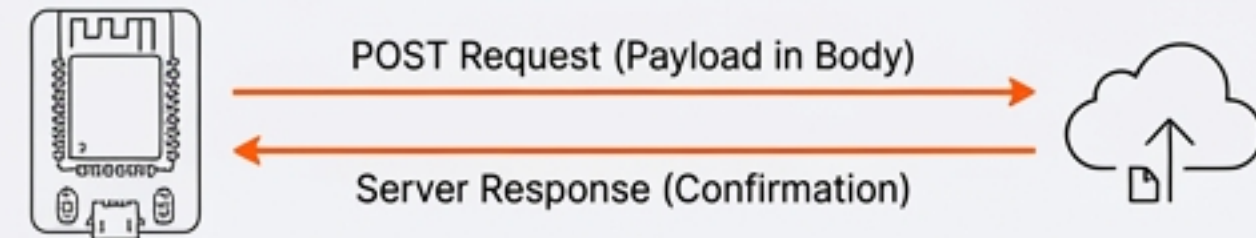
HTTP GET: Data Retrieval



Retrieves information from a server. Data is visible in the URL. Limited payload size. Ideal for fetching state, configurations, or sensor readings.

```
http://api.example.com/data?sensor_id=A1B2
```

HTTP POST: Data Submission



Sends data to a server. Data is included in the request body, not the URL. Supports larger payloads. Used for updates, commands, or sending complex data.

```
{ 'device_id': 'ESP32_001', 'temperature': 24.5, 'status': 'active' }
```

The Client-Server Ecosystem

Before code, there is conversation.
The ESP32 initiates contact; the
Server reacts.



Retrieve

Get sensor data
or configuration.

Update

Post new readings
to a database.

Trigger

Activate a remote
action.

The Great Divide: GET vs. POST

GET

The Ask



Requests data from a specified resource.

Visibility:

Visible in URL. Like shouting a question across a room—everyone can hear exactly what you asked.

POST

The Act



Sends data to create or update a resource.

Visibility:

Encapsulated in Body. Like passing a sealed note—the destination is known, but the message is hidden inside.

Anatomy of a GET Request

Transparency in the Address Bar

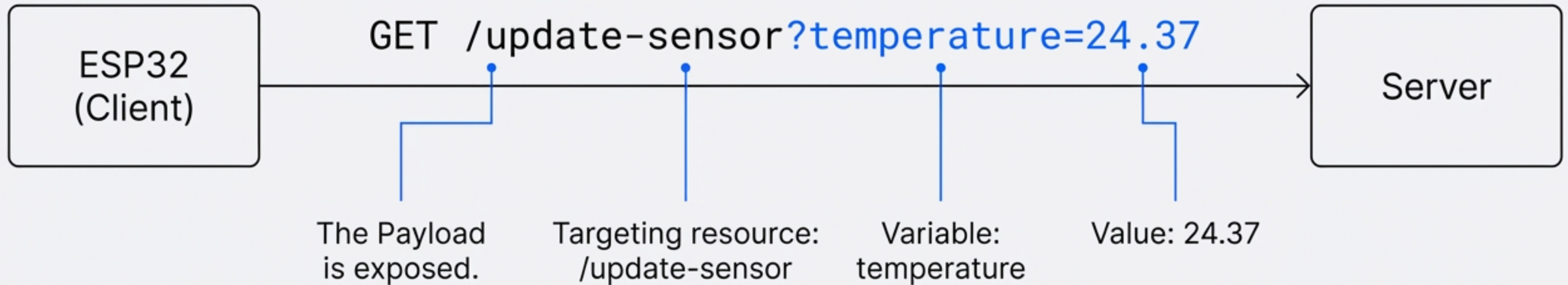


Mechanism: Data is appended directly to the URL using a '?' separator.

Transparency: Because data resides in the address bar, it is visible in network traffic logs and browser history.

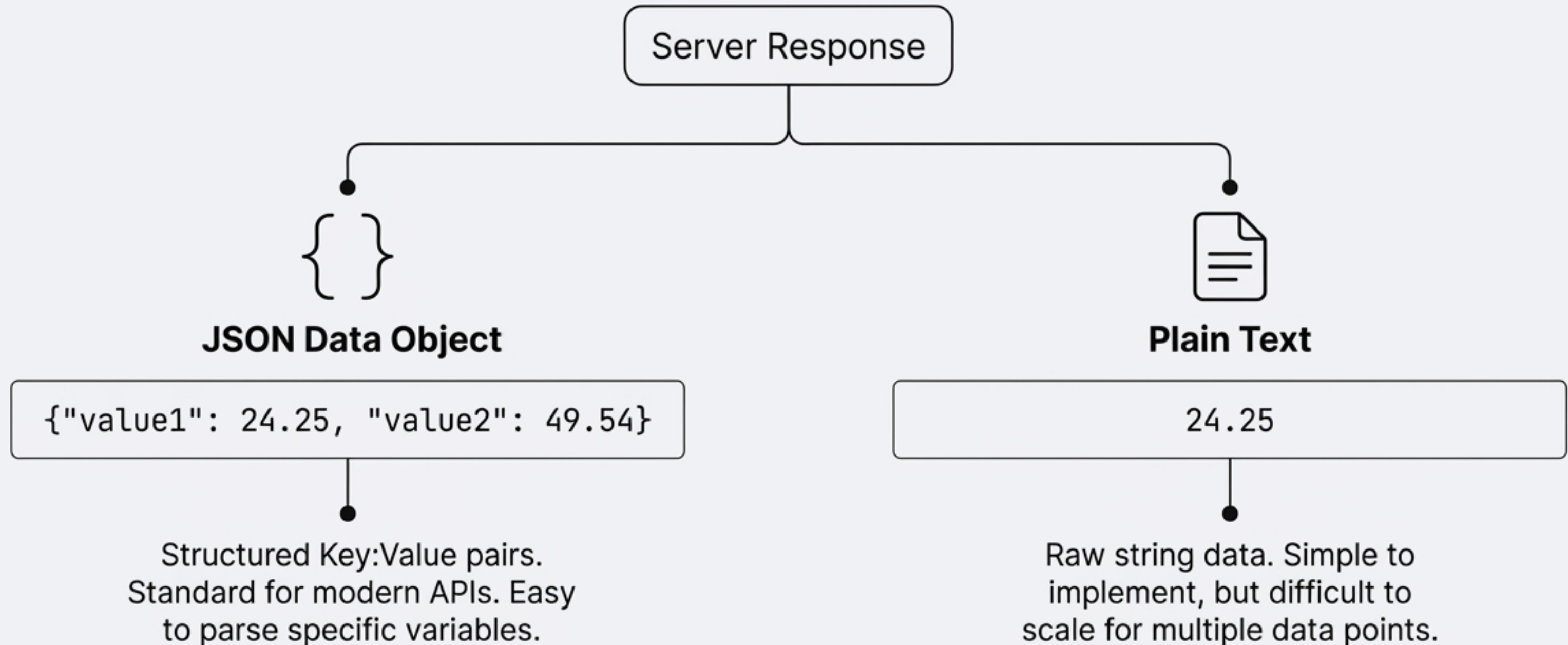
Use Case: Best for retrieving API values or lightweight, non-sensitive updates.

Visualizing GET: Data in the URL



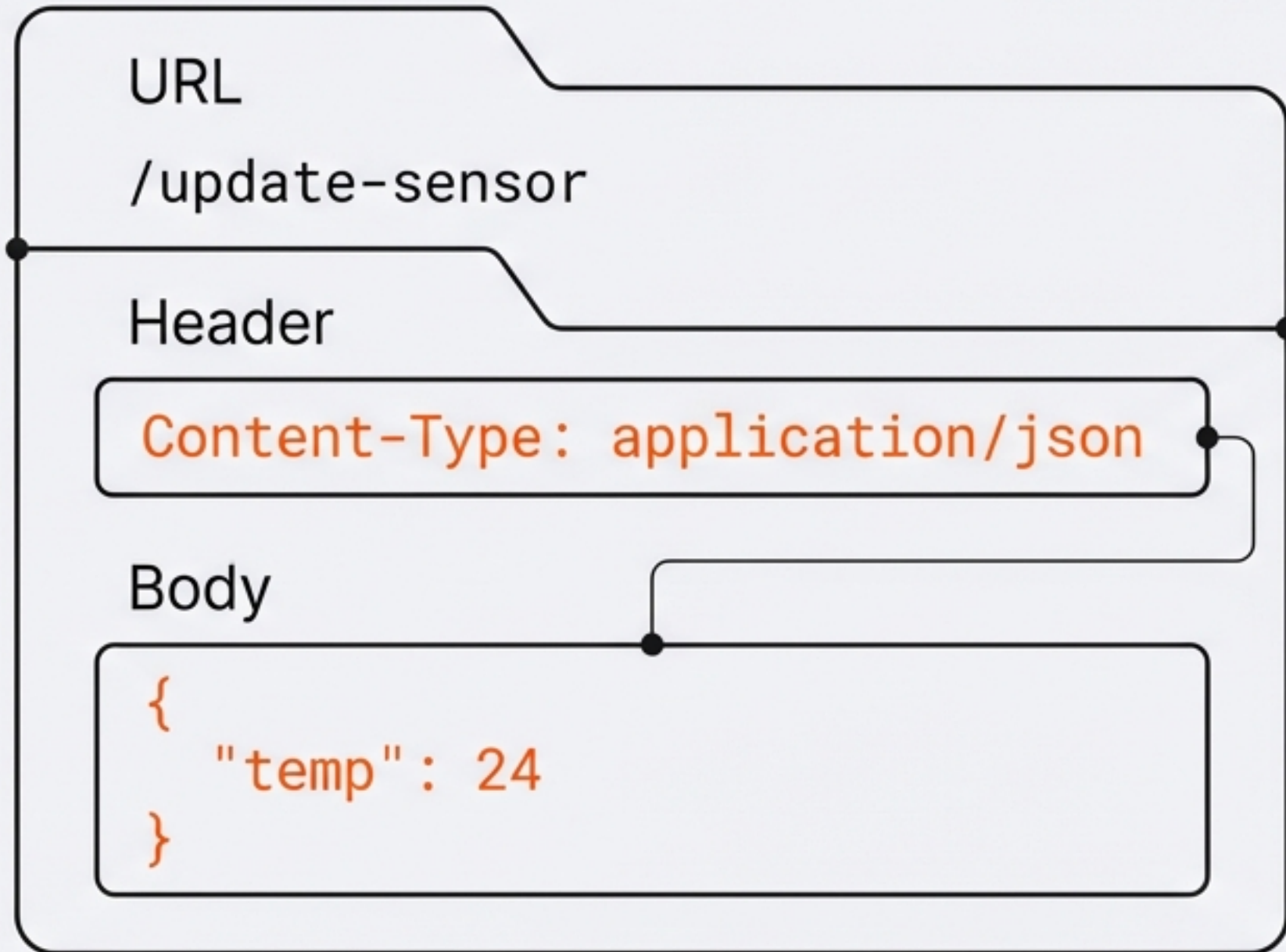
Decoding the Response

What the ESP32 receives back from the Server



Anatomy of a POST Request

Encapsulation and Headers



The Body: Unlike GET, data is separated from the URL and stored in the request body.

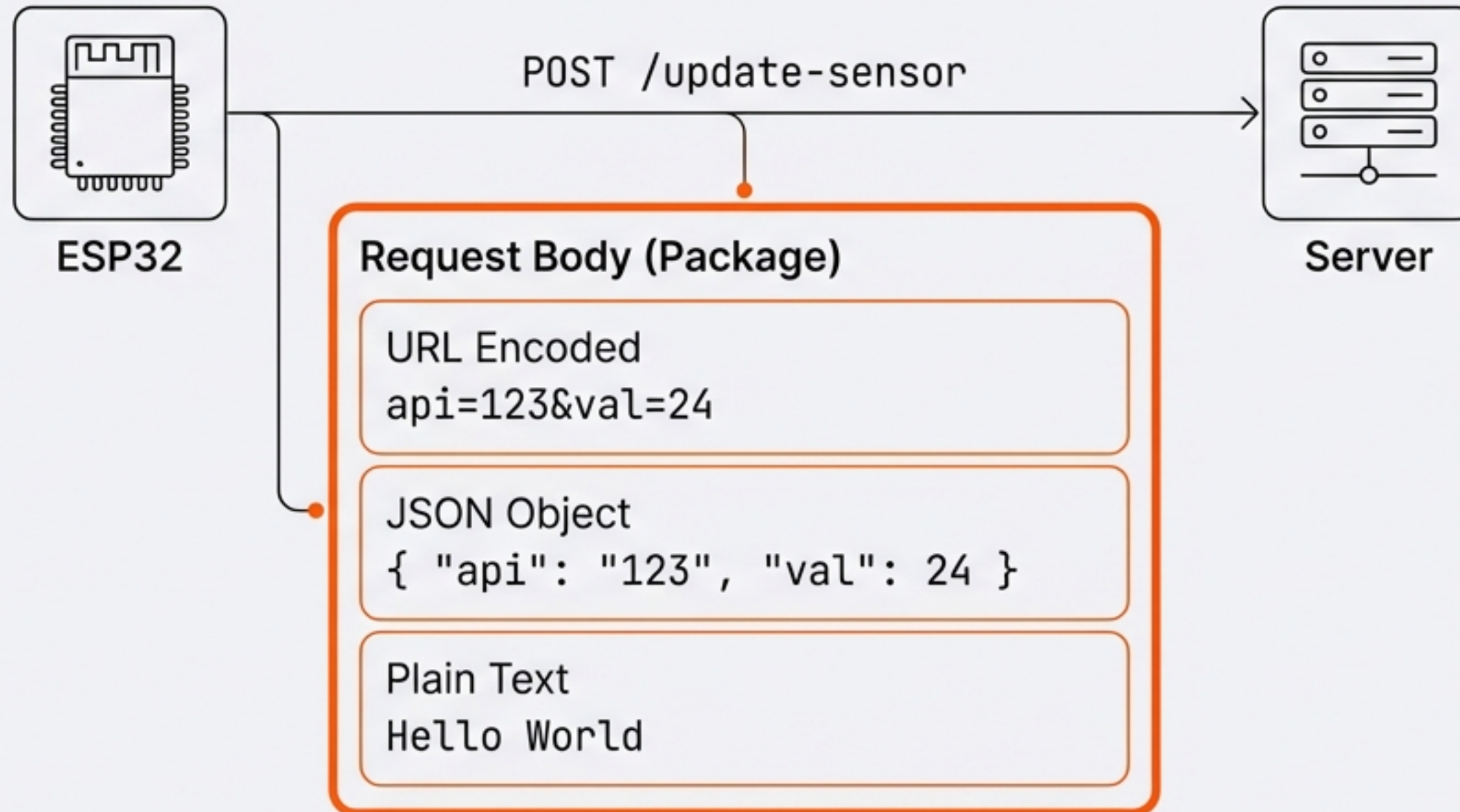
The Headers: The server must be told how to read the body. We use `'Content-Type'` to define the language (e.g., JSON).

Privacy: Data is **not** visible in the address bar.

Note: Data in the body is still readable if not encrypted via HTTPS.

Visualizing POST: Data in the Body

How data is packaged within the request



The Transport (POST) remains the same, but the Package (Body) can change format.

POST Data Formats: Choosing a Language

URL Encoded



```
api_key=123&sensor=temp
```

The traditional web form format. Good for simple key-value pairs without without hierarchy.

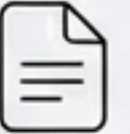
JSON Object (Industry Standard)



```
{  
  "api_key": "123",  
  "sensor": "temp"  
}
```

Highly structured, readable, and nestable. The preferred format for modern IoT APIs.

Plain Text



```
Hello World
```

Raw string. Useful for simple status messages or single values.

Summary & Best Practices

USE GET WHEN...

- You need to **READ** data.
- You are sending non-sensitive query parameters.
- You want to filter a request (e.g., `?limit=10`)

USE POST WHEN...

- You need to **WRITE** or update data.
- You are sending large payloads (like JSON).
- You want to hide parameters from the URL bar.

PRO TIP

Most modern IoT APIs (Node-RED, ThingSpeak, OpenWeatherMap) prefer **JSON payloads over HTTP POST**. This offers the highest reliability and ease of parsing for complex sensor data. Always match your Content-Type header to your data.

